AWS PYTHON WEB APPLICATION DEPLOYMENT USING DOCKER AND AWS EC2

PROJECT TITLE

AWS Python Web Application Deployment Using Docker and AWS EC2

Technologies Used:
Python, Flask, Docker, GitHub, AWS EC2, AWS ECR

AWS Region:
ap-south-1 (Mumbai)

Submitted By:
Krishna Kumar

Project Type:
Cloud / DevOps Deployment Project

Date:
September 2026

ABSTRACT

This project demonstrates the development and deployment of a simple Python web application using Flask. The application is containerized using Docker and deployed on an AWS EC2 instance.

The source code is maintained in a GitHub repository. Docker is used to package the application and its dependencies into a portable container. The Docker container is then executed on an Ubuntu-based AWS EC2 instance.

The project also demonstrates the use of AWS Elastic Container Registry (ECR) for storing Docker images and provides a basic understanding of cloud-based application deployment.

INTRODUCTION

Cloud computing allows applications to be deployed and accessed over the internet without depending on local systems.

In this project, a simple Python Flask web application is developed and containerized using Docker. The source code is stored in GitHub, while AWS EC2 is used as the cloud server for running the Docker container.

The project demonstrates the complete flow:

Developer → GitHub → Docker → AWS EC2 → Web Application

The application provides a web page and REST API endpoints that can be accessed through a web browser

OBJECTIVES

The main objectives of this project are:

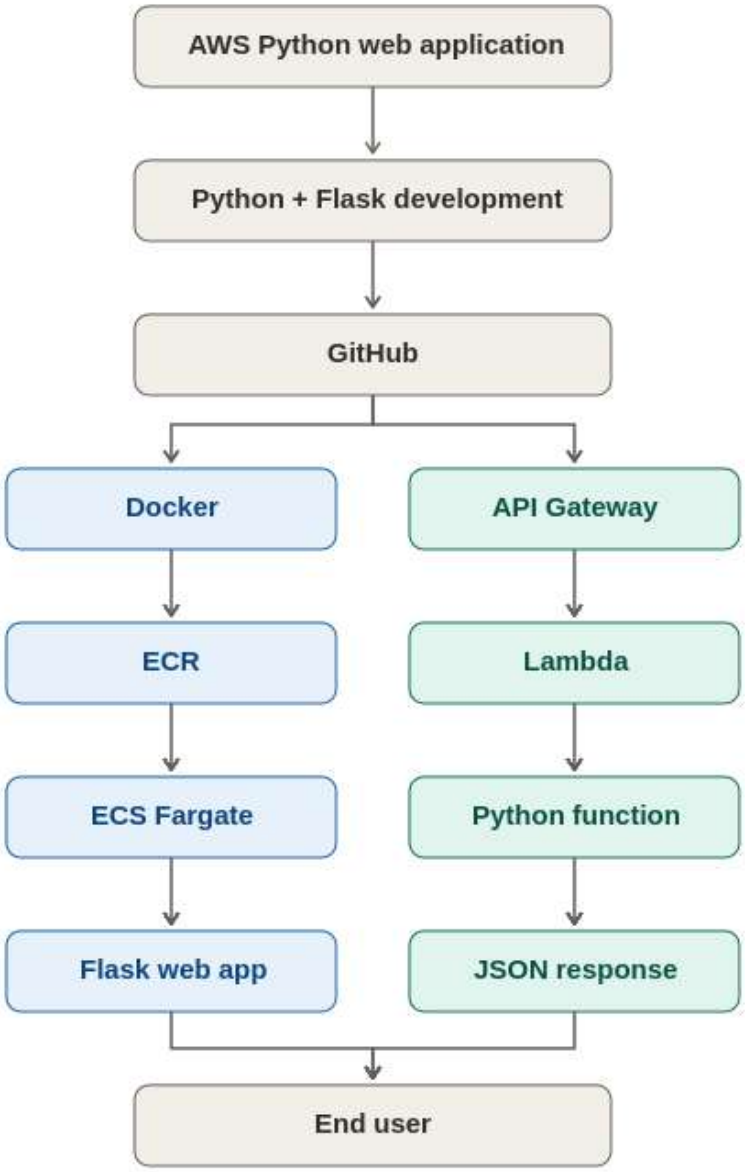
1. To develop a basic Python Flask web application.
2. To create a simple HTML frontend.
3. To create REST API endpoints using Flask.
4. To store the project source code in GitHub.
5. To create a Dockerfile for containerization.
6. To build a Docker image.
7. To run the application inside a Docker container.
8. To create and use an AWS EC2 instance.
9. To deploy the Docker container on EC2.
10. To understand AWS ECR for Docker image storage.
11. To access and test the application through the EC2 public IP address.

TECHNOLOGIES USED

Technology	Purpose
Python	Application development
Flask	Python web framework
HTML	Frontend development
JavaScript	API interaction
Docker	Application containerization
Git	Version control
GitHub	Source code repository
AWS EC2	Cloud server
AWS ECR	Docker image registry
Ubuntu	EC2 operating system
PowerShell	Local command-line environment

SYSTEM ARCHITECTURE

The system architecture shows how the Python Flask application moves from local development to cloud deployment. The application is first developed and tested locally. It is then containerized using Docker and stored in GitHub. Finally, the Dockerized application is deployed on an AWS EC2 Ubuntu server.



Architecture Explanation

The developer creates the Python Flask application and pushes the source code to GitHub.

The application is packaged into a Docker image using the Dockerfile. The Docker image can be stored in AWS ECR.

For deployment, the application is run inside a Docker container on an AWS EC2 Ubuntu server.

Users can access the application using:

```
http://EC2_PUBLIC_IP:5000
```

PROJECT STRUCTURE

The project contains the following files:

```
aws-python-webapp/
│
├── app.py
├── requirements.txt
├── Dockerfile
│
└── templates/
    └── index.html
```

File Description

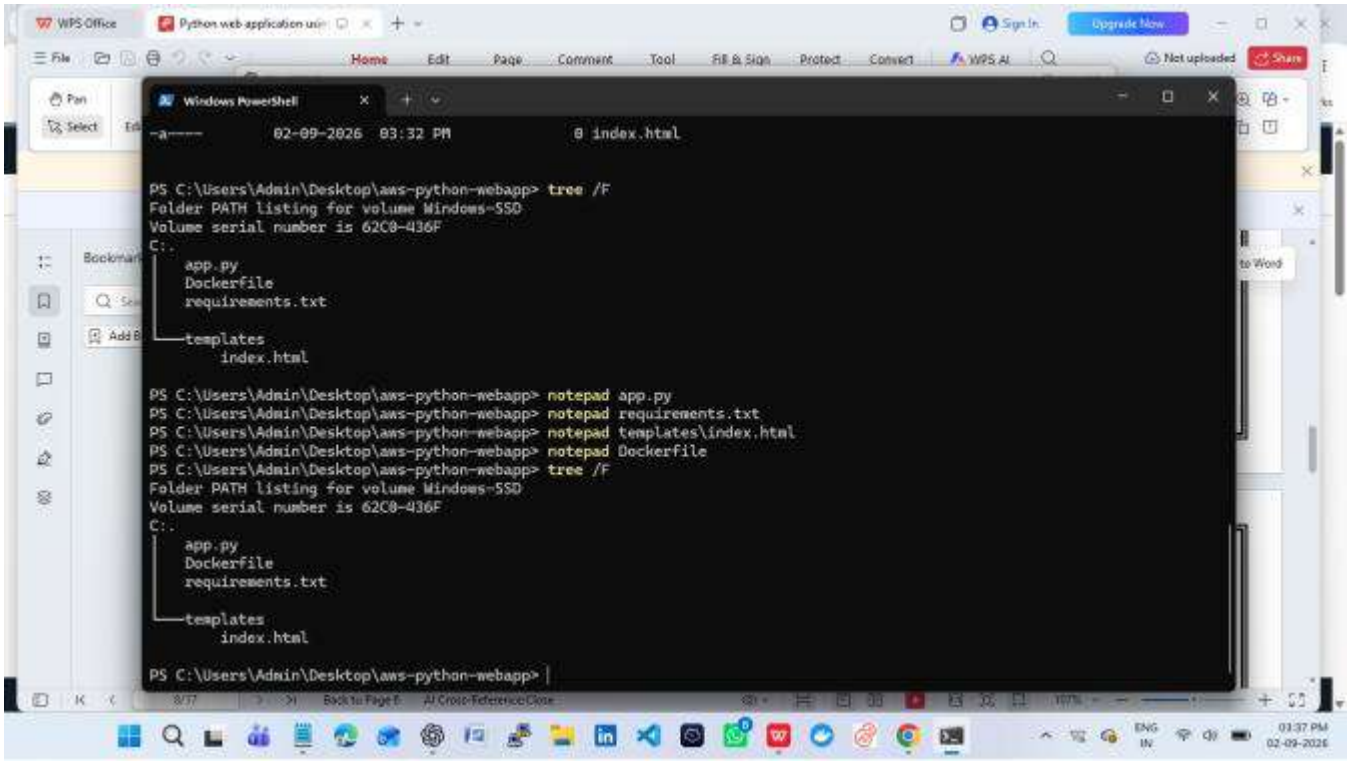
app.py
Contains the Flask application and API routes.

requirements.txt
Contains the Python dependencies required by the application.

Dockerfile
Contains instructions for creating the Docker image.

templates/index.html
Contains the frontend HTML page

Project Folder Structure



Create the Python Flask Application

Create the project folder and templates directory in PowerShell:

```
mkdir aws-python-webapp
cd aws-python-webapp
mkdir templates
```

Create app.py:

```
from flask import Flask, jsonify, render_template

app = Flask(__name__)

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/api/status")
def status():
    return jsonify({
        "message": "AWS Python Web App is running",
        "application": "Python Flask Web App",
        "status": "success"
    })

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

Create requirements.txt:

```
Flask==3.1.2
```

Create templates/index.html:

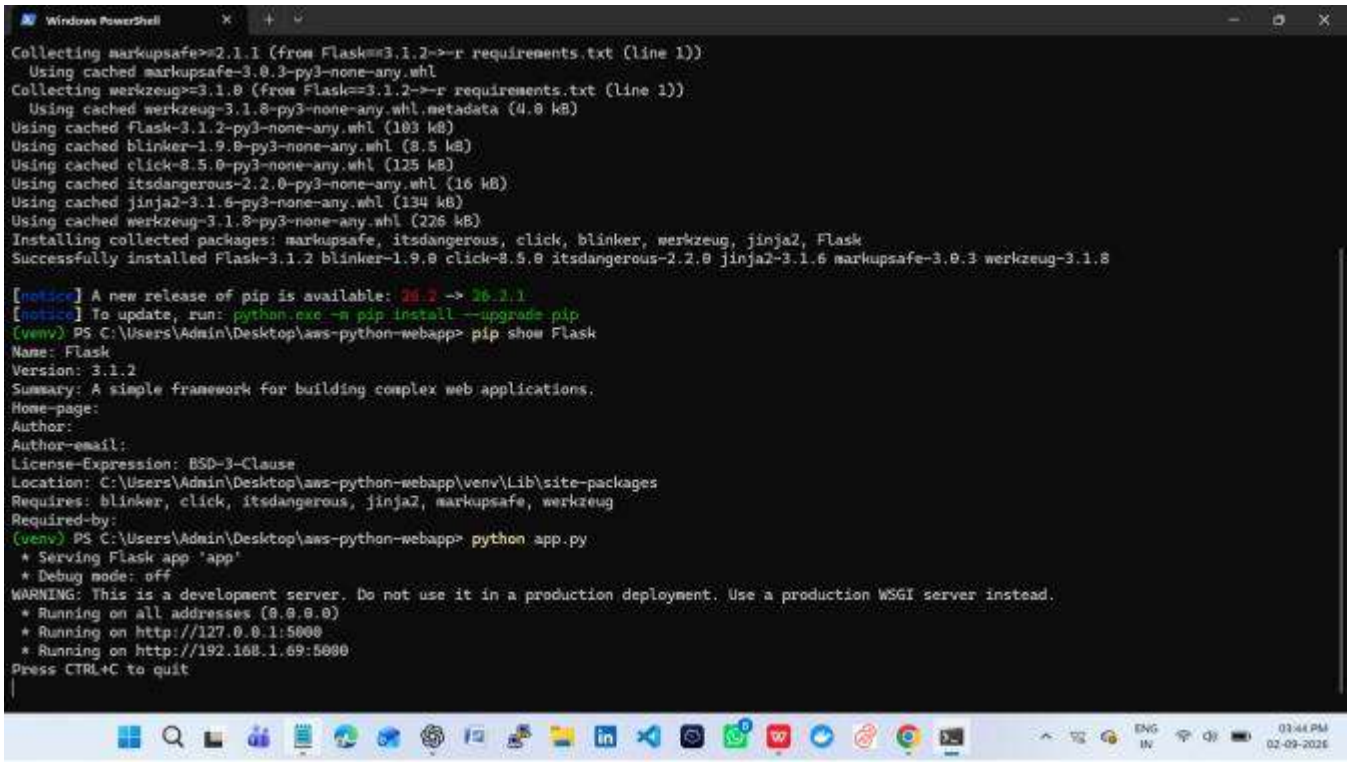
```
<!DOCTYPE html>
<html>
<head>
    <title>AWS Python Web App</title>
</head>
<body>
    <h1>AWS Python Web App</h1>
    <p>Application is running successfully.</p>
    <p>Deployment: Python Flask</p>
</body>
</html>
```

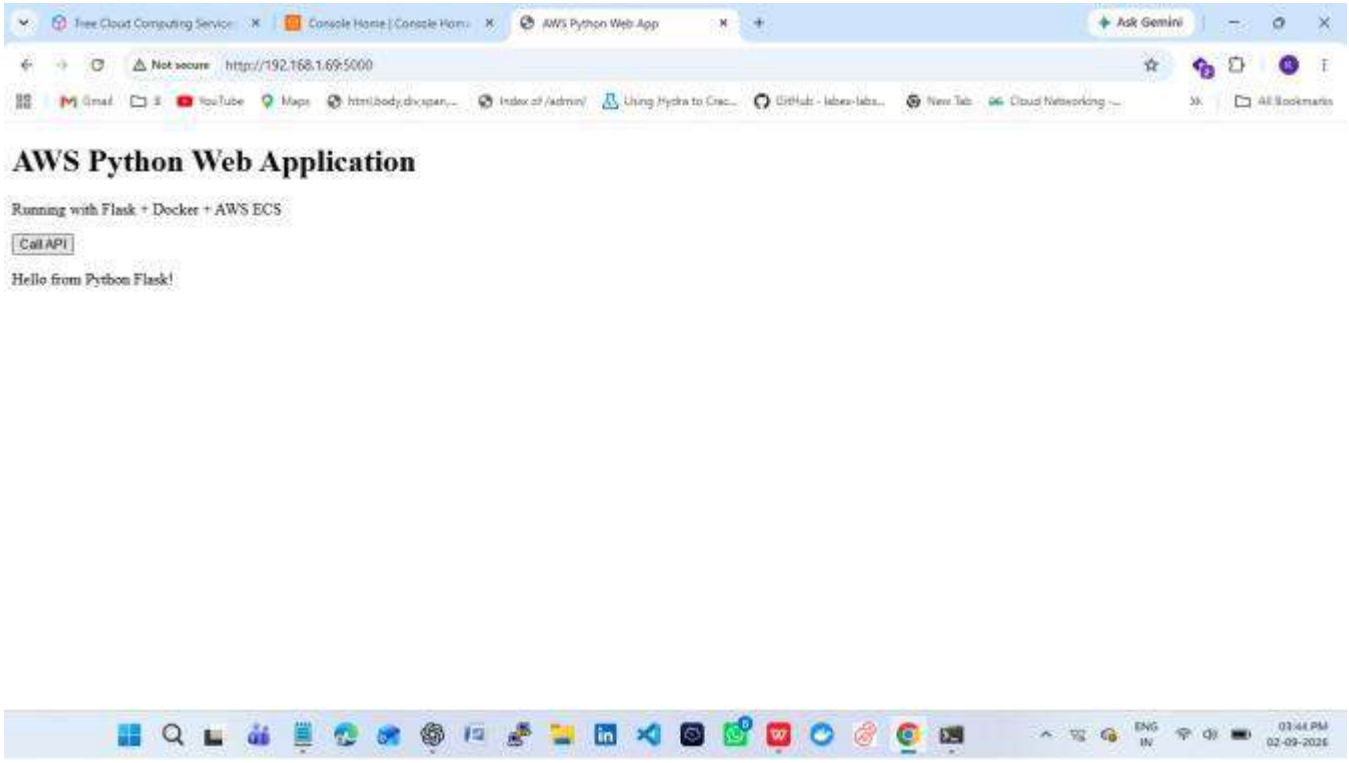
Test the Application Locally

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Open the browser: http://127.0.0.1:5000/

API test: http://127.0.0.1:5000/api/statusExpected API status: application = Python Flask Web App, status = success.



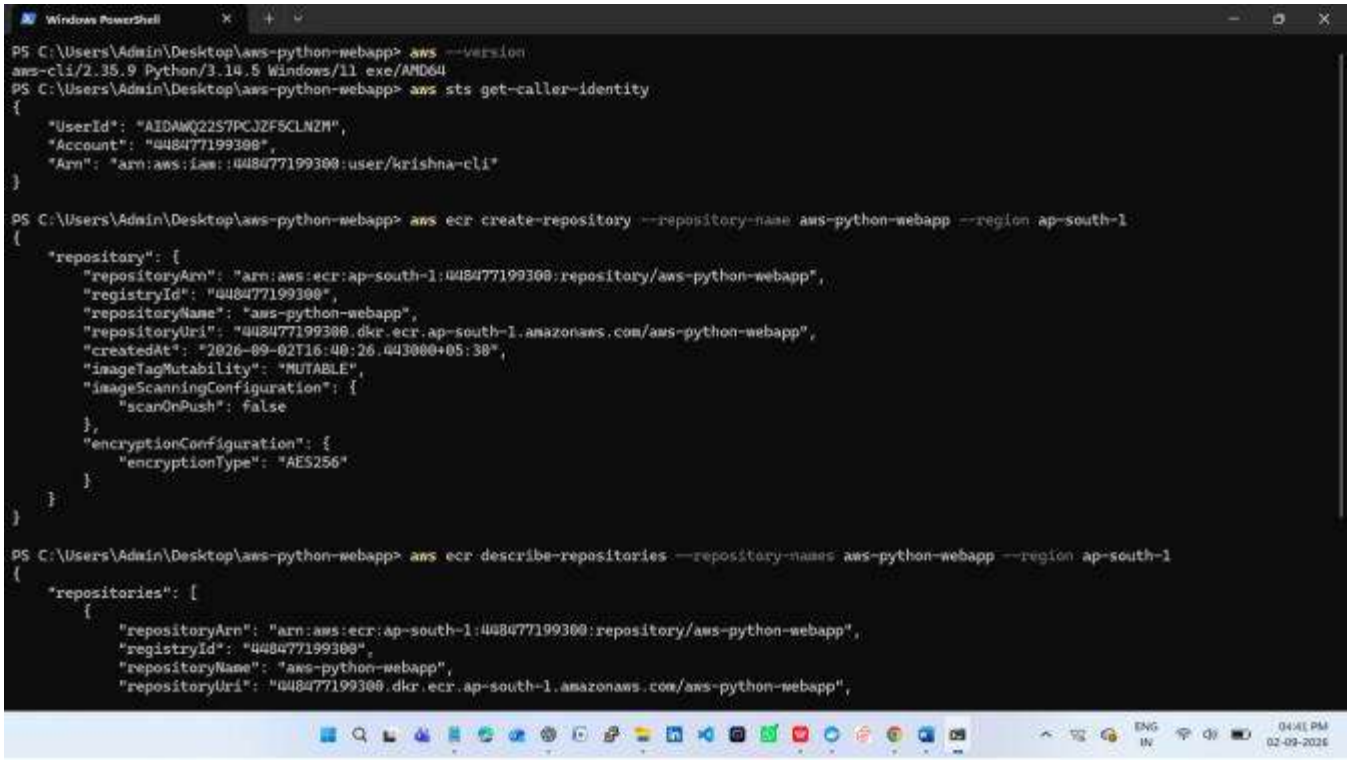

Free Cloud Computing Service
Console Home | Console Home
AWS Python Web App
Ask Gemini
Not secure
http://192.168.1.69:5000
Gmail
YouTube
Maps
Index of /admin/
Using Hydra to Crac...
GitHub - labex-labs...
New Tab
Cloud Networking -...
All Bookmarks
AWS Python Web Application
Running with Flask + Docker + AWS ECS
Call API
Hello from Python Flask!
ENG
IN
03:44 PM
02-09-2026

Configure AWS CLI

```
aws configure
aws sts get-caller-identity
```

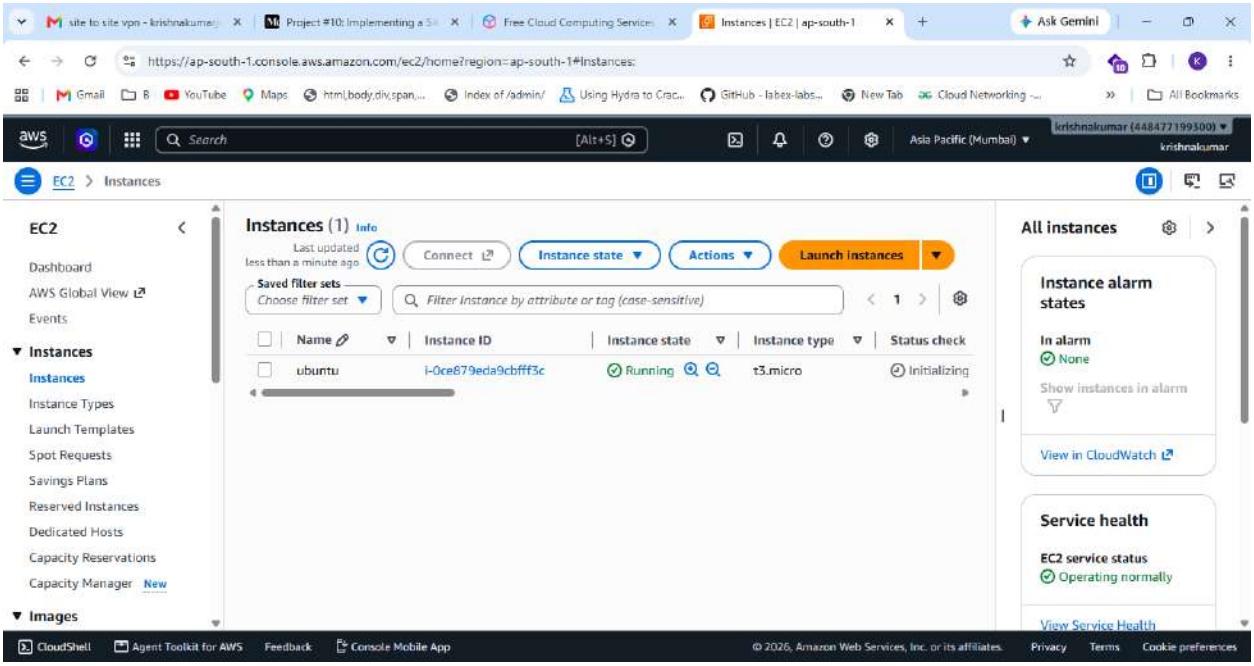
Enter the AWS credentials, default region (ap-south-1), and output format (json) when prompted.

Security note: Never place AWS access keys, secret keys, passwords, or private keys in source code, Dockerfiles, GitHub, or screenshots.



Launch an Ubuntu EC2 Instance

1. Open AWS Console → EC2 → Instances → Launch instance.
2. Name: aws-python-webapp-ec2.
3. Choose an Ubuntu Server LTS AMI.
4. Choose a small eligible instance type suitable for your account/budget.
5. Create/select a key pair and download the .pem file.
6. Security Group: SSH TCP 22 from your IP only.
7. For direct Flask testing, allow TCP 5000 from your IP.
8. Launch the instance and copy its public IPv4 address.

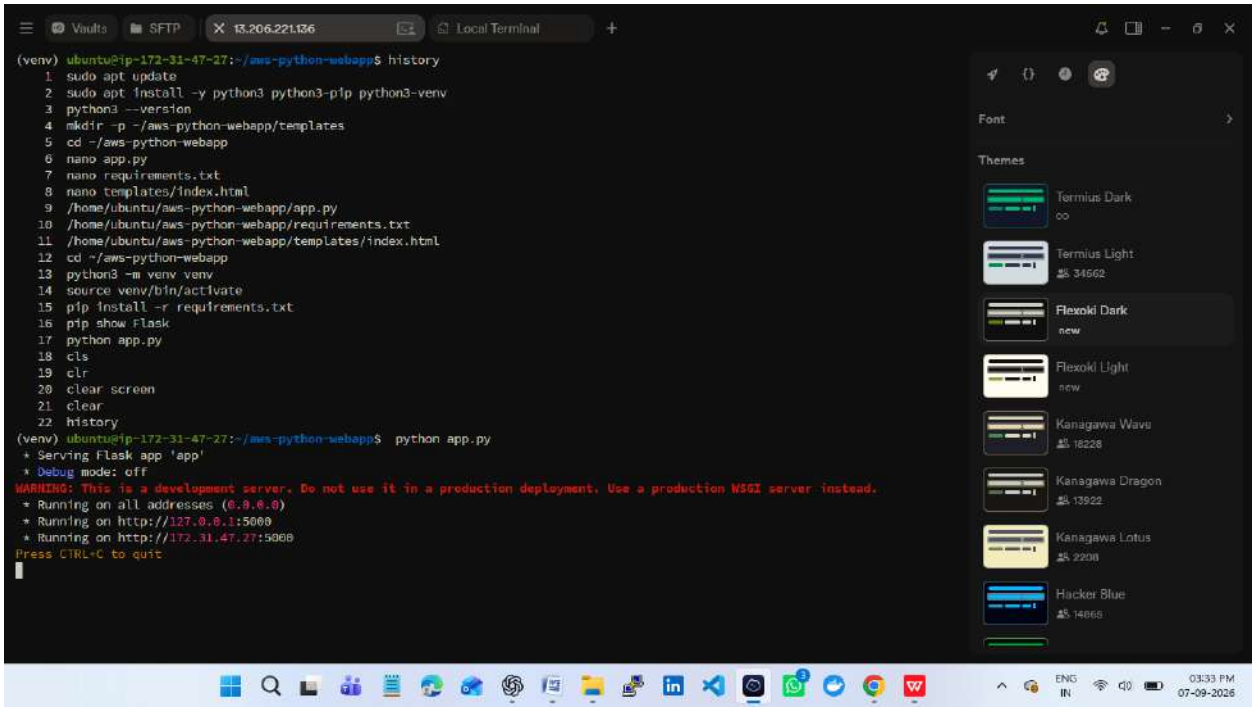


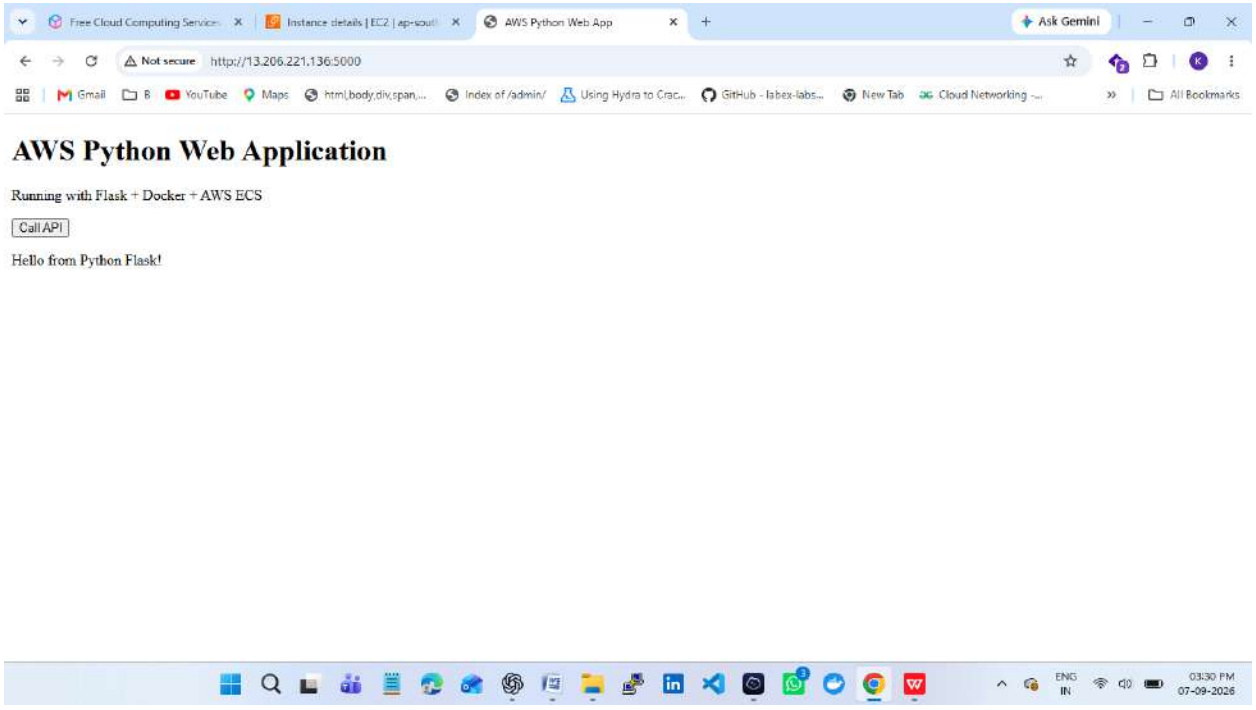
Install Python and Run the App on EC2

```
sudo apt update
sudo apt install -y python3 python3-venv python3-pip
mkdir -p ~/aws-python-webapp
cd ~/aws-python-webapp
```

Copy the project files to EC2 using Git, SCP, or another secure method. Then run:

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python app.pyOpen: http://<EC2_PUBLIC_IP>:5000/
```




Free Cloud Computing Services
Instance details | EC2 | ap-south
AWS Python Web App
Ask Gemini
Not secure
http://13.206.221.136:5000
Gmail
B
YouTube
Maps
html,body,div,span,...
Index of /admin/
Using Hydra to Crac...
GitHub - labex-labs...
New Tab
Cloud Networking -...
All Bookmarks
AWS Python Web Application
Running with Flask + Docker + AWS ECS
Call API
Hello from Python Flask!
ENG
IN
03:30 PM
07-09-2026

Create the Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

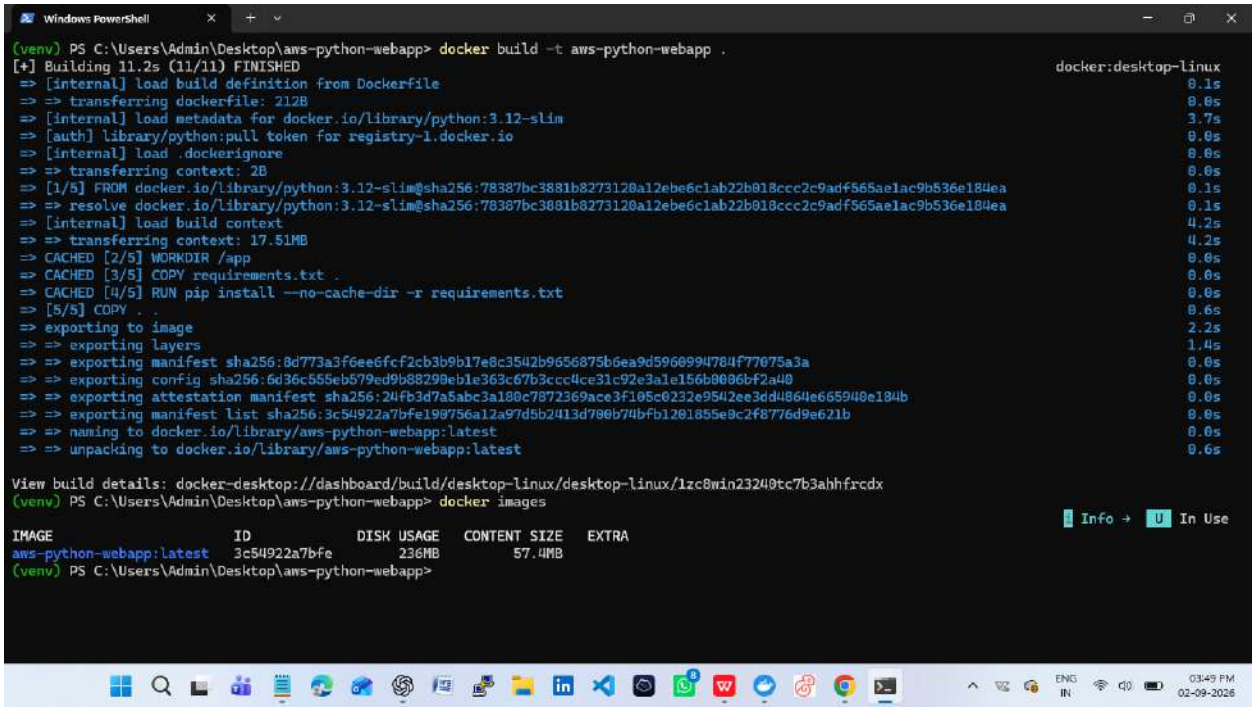
Save the file exactly as Dockerfile with no .txt extension.

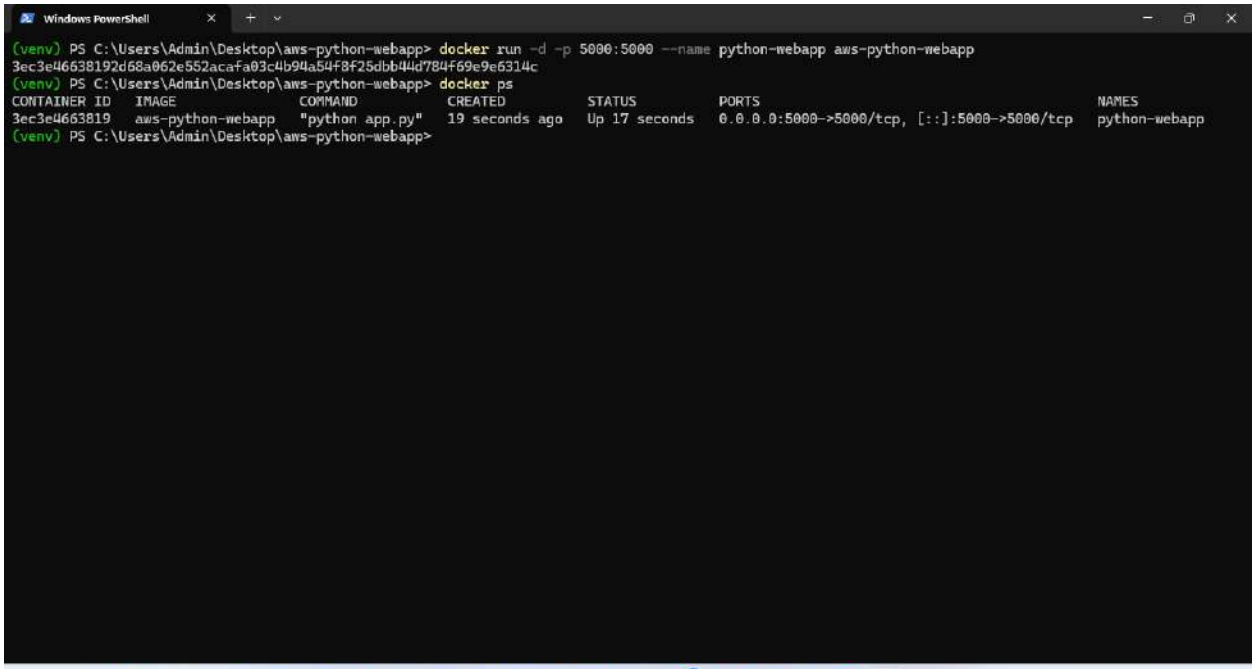
Build and Test the Docker Image

```
docker build -t aws-python-webapp .
docker images
docker run -d --name aws-python-webapp-container -p 5000:5000 aws-python-webapp
docker ps
```

Open: http://localhost:5000/

```
docker stop aws-python-webapp-container
docker rm aws-python-webapp-container
```





AWS Python Web Application

Running with Flask + Docker + AWS ECS

Call API

Hello from Python Flask!

Create Amazon ECR Repository

AWS Console → ECR → Private repositories → Create repository.

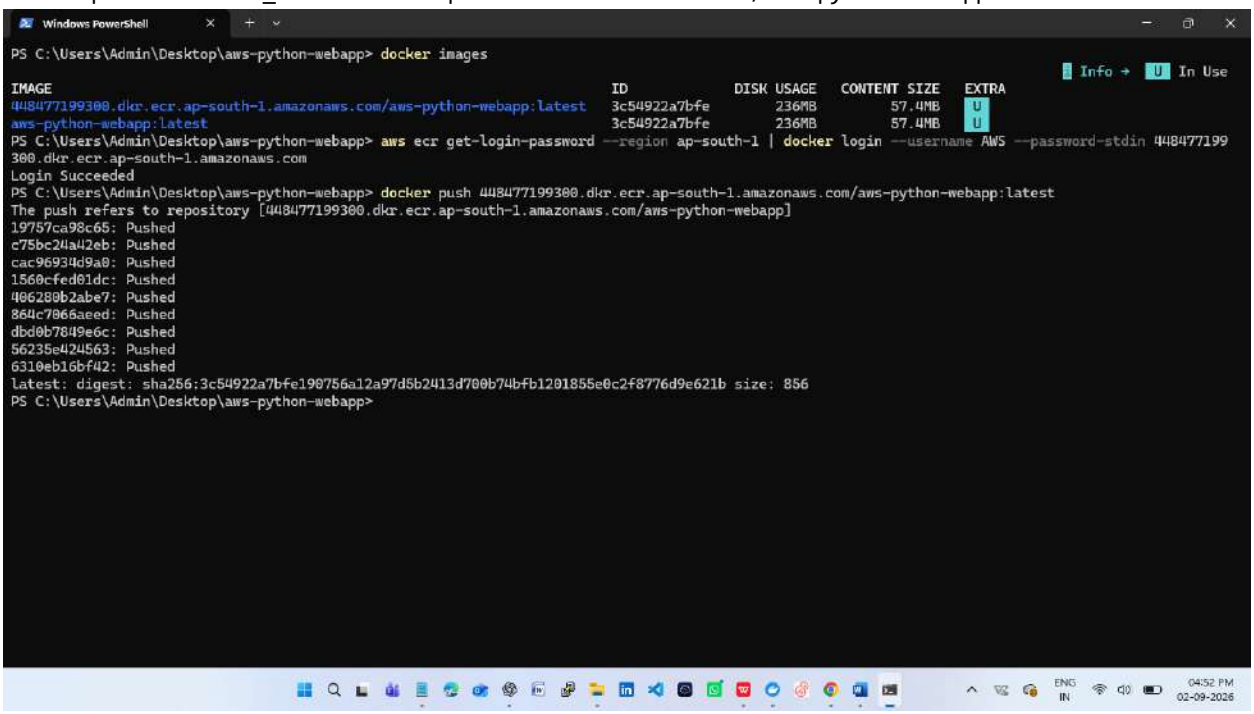
Repository name: aws-python-webapp

Push Docker Image to ECR

```
aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com

docker tag aws-python-webapp:latest <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest

docker push <ACCOUNT_ID>.dkr.ecr.ap-south-1.amazonaws.com/aws-python-webapp:latest
```

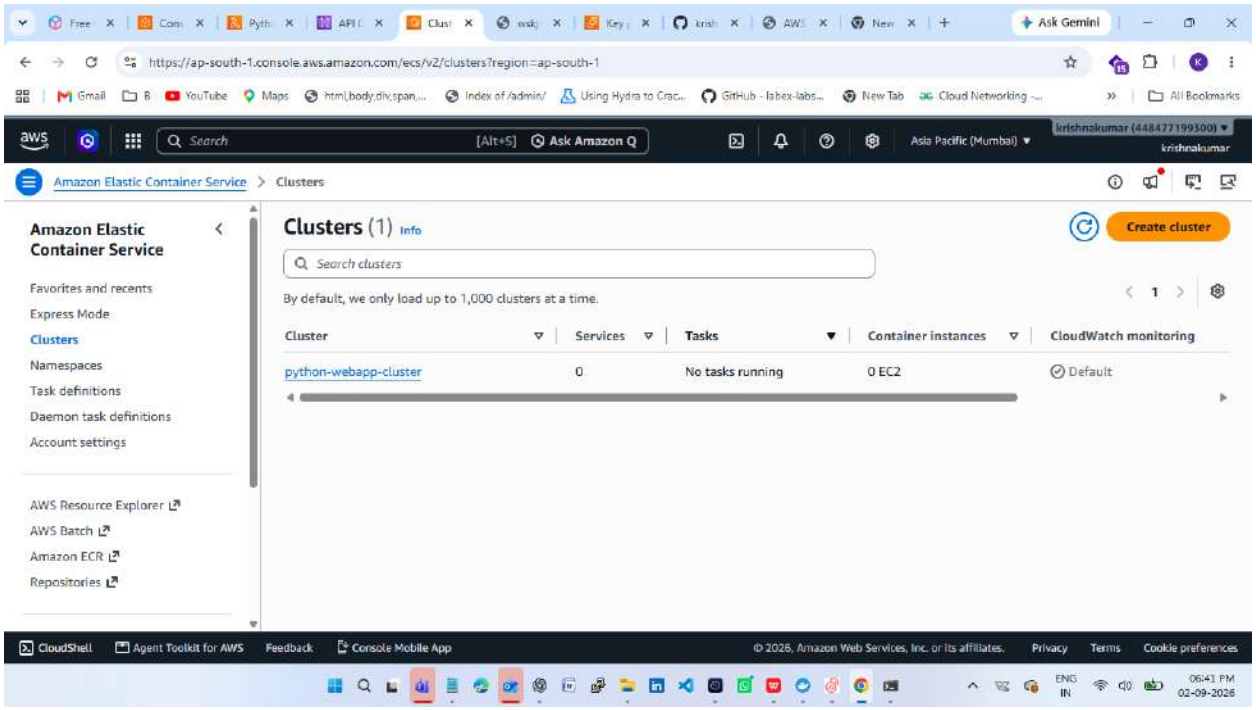


Create ECS Cluster

AWS Console → ECS → Clusters → Create cluster.

Cluster name: aws-python-webapp-cluster

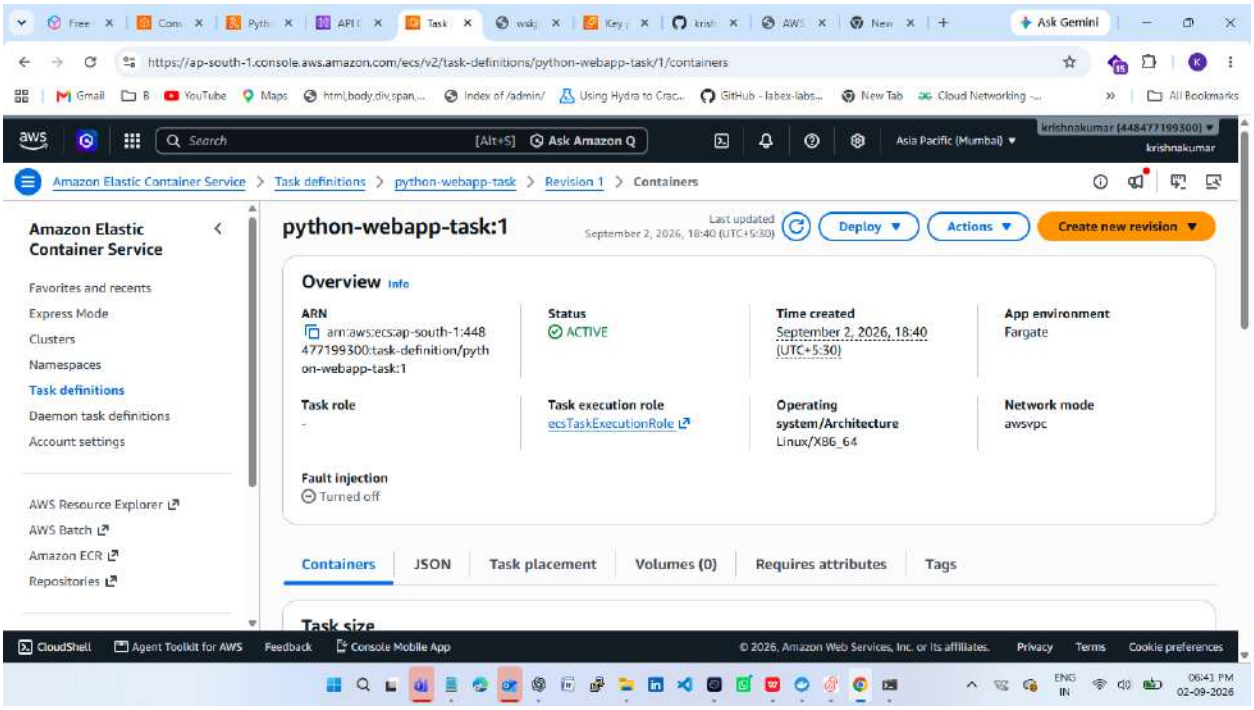
Use ECS with AWS Fargate so the container can run without managing an ECS EC2 worker fleet.



.

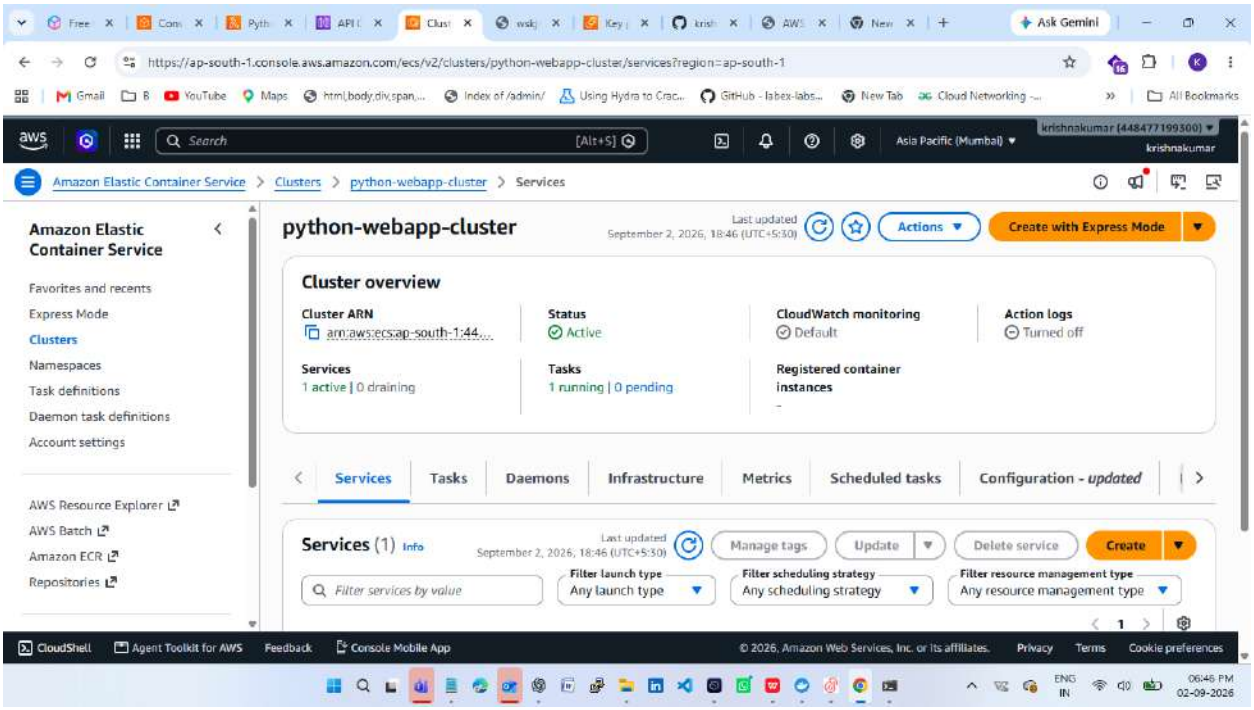
Create ECS Task Definition

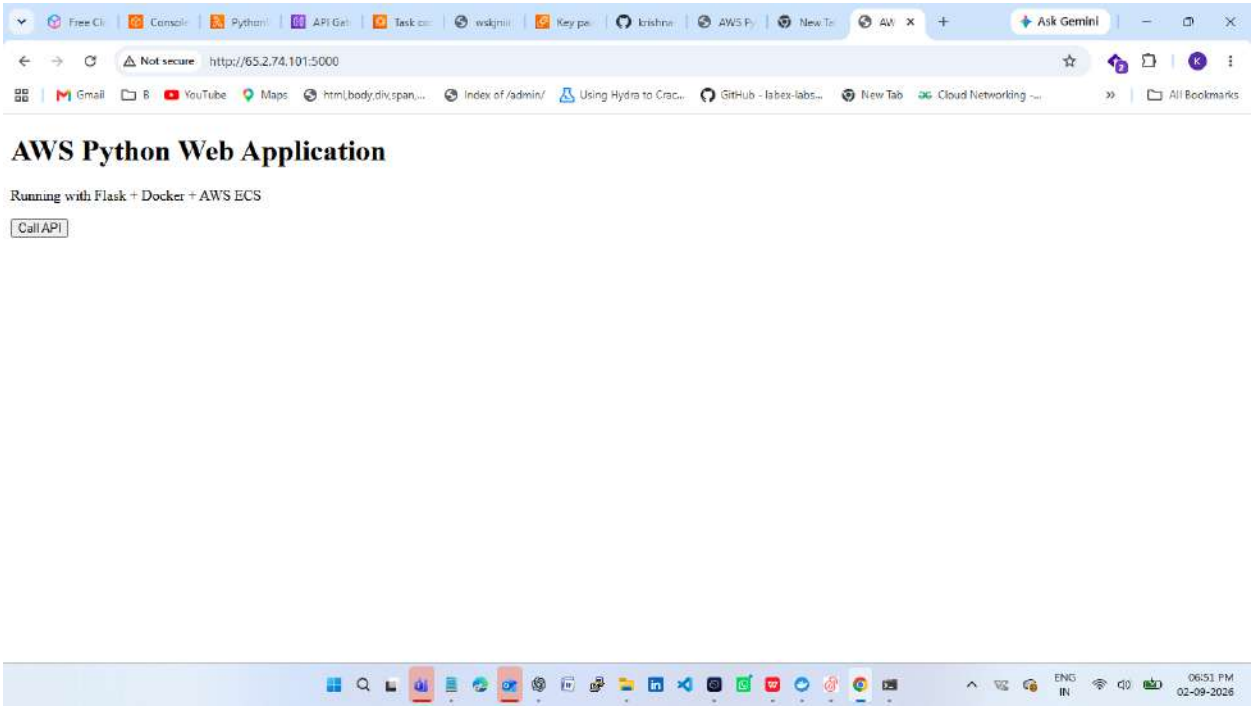
- Family: aws-python-webapp-task.
- Compatibility: Fargate.
- CPU: 0.25 vCPU / 256 CPU units.
- Memory: 0.5 GB / 512 MiB.
- Task execution role: ECS task execution role with AmazonECSTaskExecutionRolePolicy.
- Container name: aws-python-webapp.
- Image URI: your ECR image URI ending in :latest.
- Container port: 5000 TCP.



Create ECS Service

1. Open ECS → Clusters → aws-python-webapp-cluster → Create service.
2. Compute option: Fargate.
3. Task definition: aws-python-webapp-task.
4. Service name: aws-python-webapp-service.
5. Desired tasks: 1.
6. Select the VPC and public subnets.
7. Allow TCP 5000 from your IP for learning/testing, or from a load balancer security group if an ALB is used.
8. Enable public IP for direct testing without a load balancer.
9. Create the service.


https://ap-south-1.console.aws.amazon.com/ecs/v2/clusters/python-webapp-cluster/services?region=ap-south-1
Search
Asia Pacific (Mumbai)
krishnakumar
Amazon Elastic Container Service > Clusters > python-webapp-cluster > Services
Amazon Elastic Container Service
Favorites and recents
Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions
Account settings
AWS Resource Explorer
AWS Batch
Amazon ECR
Repositories
python-webapp-cluster
Last updated September 2, 2026, 18:46 (UTC+5:30)
Actions
Create with Express Mode
Cluster overview
Cluster ARN
arn:aws:ecs:ap-south-1:44...
Status
Active
CloudWatch monitoring
Default
Action logs
Turned off
Services
1 active | 0 draining
Tasks
1 running | 0 pending
Registered container instances
Services
Tasks
Daemons
Infrastructure
Metrics
Scheduled tasks
Configuration - updated
Services (1) Info
Last updated September 2, 2026, 18:46 (UTC+5:30)
Manage tags
Update
Delete service
Create
Filter services by value
Filter launch type
Any launch type
Filter scheduling strategy
Any scheduling strategy
Filter resource management type
Any resource management type
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
06:46 PM
02-09-2026


Not secure
http://65.2.74.101:5000
AWS Python Web Application
Running with Flask + Docker + AWS ECS
Call API
06:51 PM
02-09-2026

Lambda Function (Serverless Demonstration)

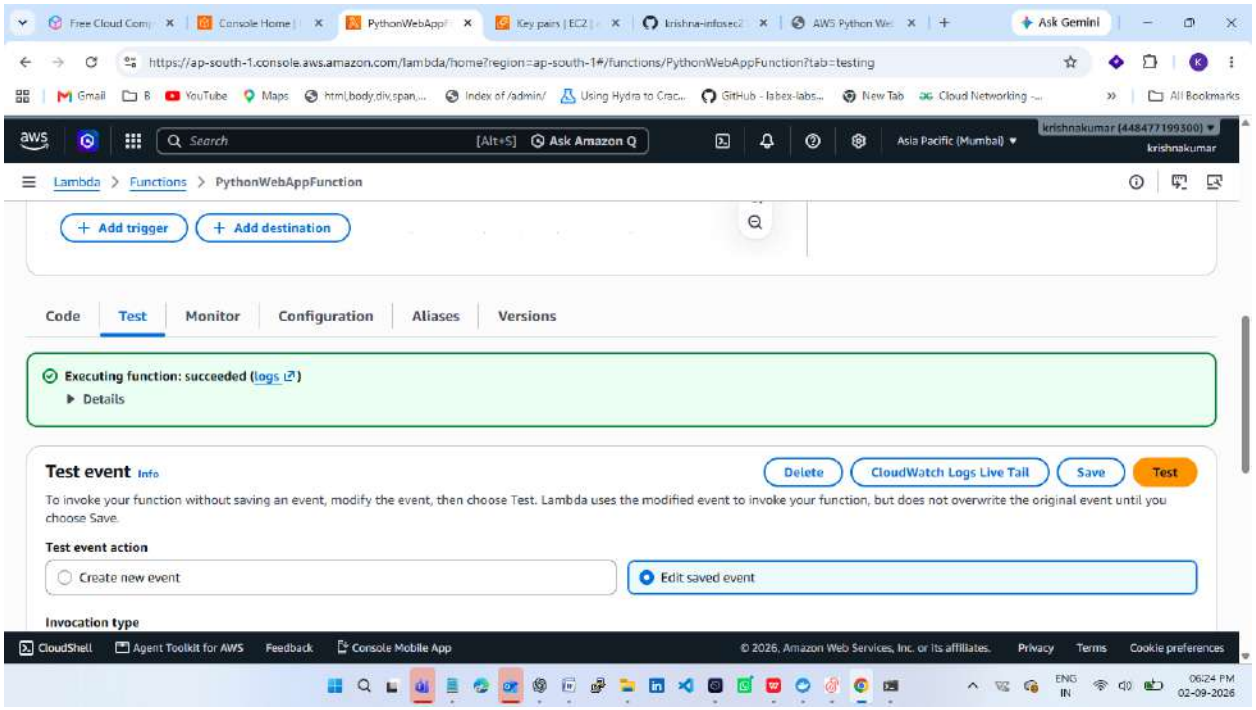
AWS Console → Lambda → Create function → Author from scratch.

Function name: aws-python-webapp-lambda. Choose a currently supported Python runtime.

```
import json

def lambda_handler(event, context):
    return {
        "statusCode": 200,
        "headers": {
            "Content-Type": "application/json"
        },
        "body": json.dumps({
            "message": "Hello from AWS Lambda!",
            "application": "Python Web App",
            "status": "success"
        })
    }
```

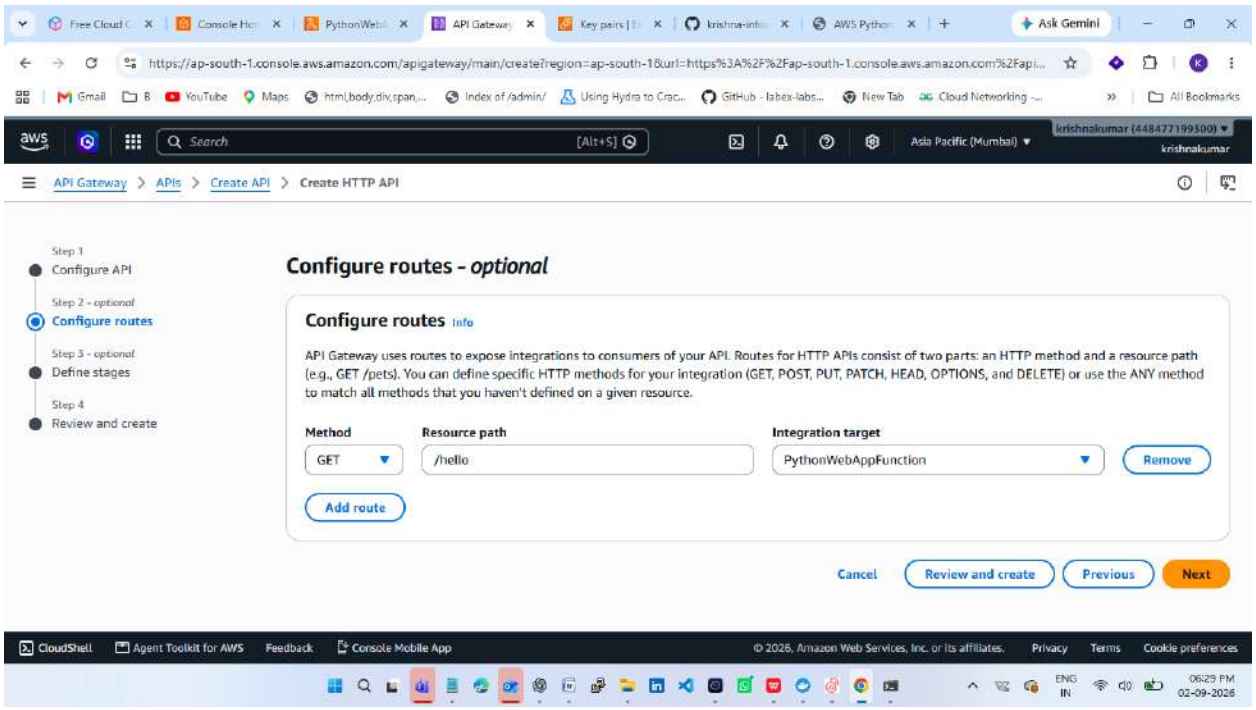
Choose Deploy and test the function using the Lambda Test tab.

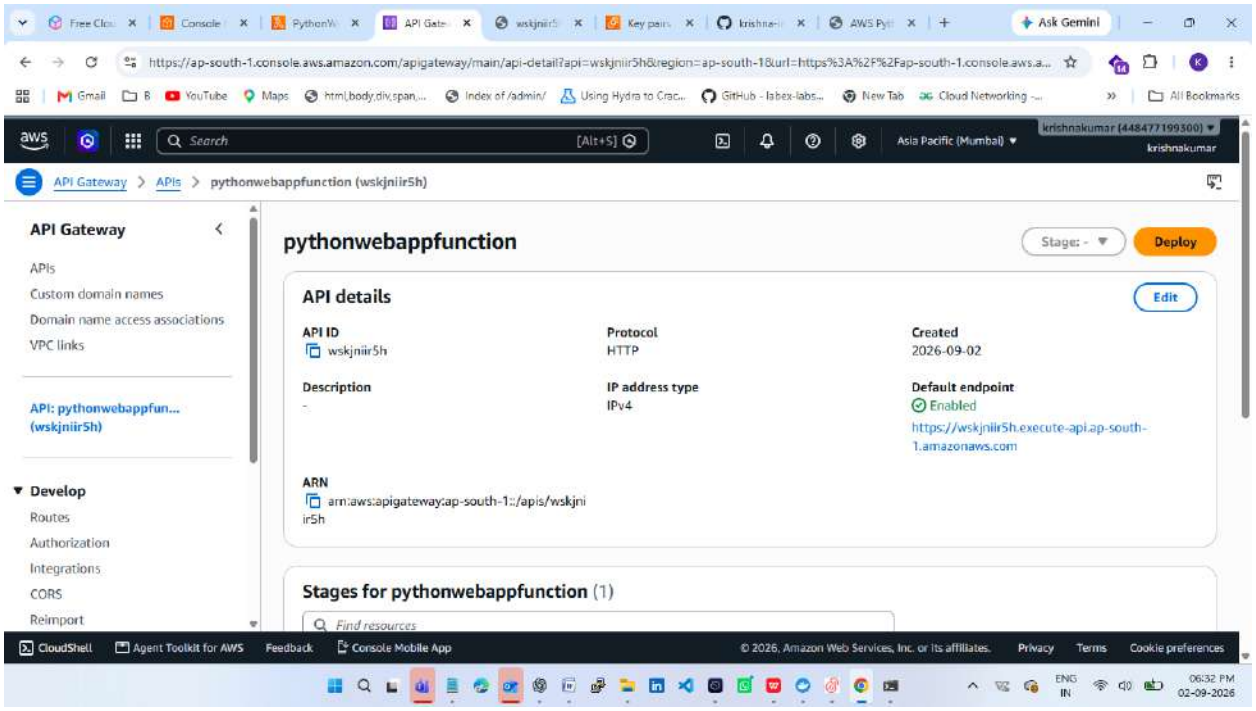


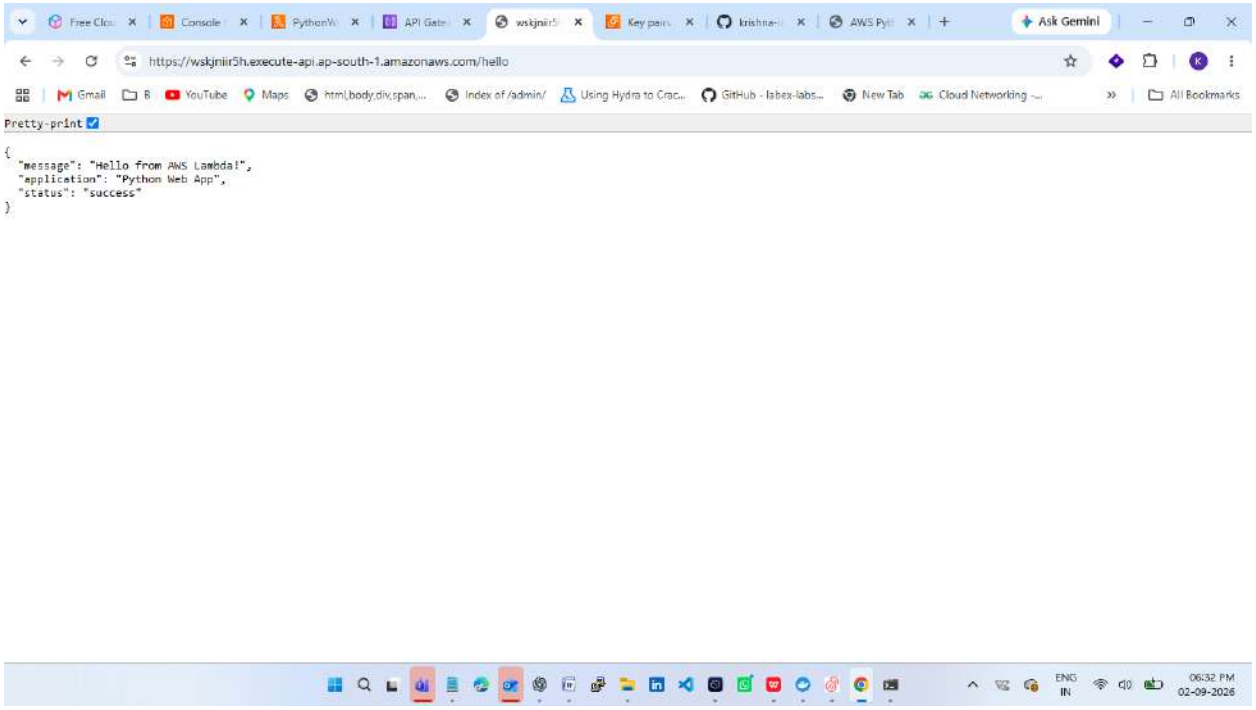
API Gateway HTTP API

1. Open API Gateway → APIs → Create API.
2. Under HTTP API, choose Build.
3. Add Lambda integration and select aws-python-webapp-lambda.
4. API name: aws-python-webapp-api.
5. Route: GET /hello.
6. Create the API and copy the Invoke URL.

Test: <API_INVOKE_URL>/hello




Ask Gemini
https://ap-south-1.console.aws.amazon.com/apigateway/main/api-detail?api=wskjniir5h&region=ap-south-1&url=https%3A%2F%2Fap-south-1.console.aws.a...
Search
[Alt+S]
Asia Pacific (Mumbai)
krishnakumar (448477199300)
krishnakumar
API Gateway > APIs > pythonwebappfunction (wskjniir5h)
API Gateway
APIs
Custom domain names
Domain name access associations
VPC links
API: pythonwebappfun... (wskjniir5h)
Develop
Routes
Authorization
Integrations
CORS
Reimport
pythonwebappfunction
Stage: -
Deploy
API details
Edit
API ID
wskjniir5h
Protocol
HTTP
Created
2026-09-02
Description
-
IP address type
IPv4
Default endpoint
Enabled
https://wskjniir5h.execute-api.ap-south-1.amazonaws.com
ARN
arn:aws:apigateway:ap-south-1::/apis/wskjniir5h
Stages for pythonwebappfunction (1)
Find resources
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
ENG
IN
06:32 PM
02-09-2026


Ask Gemini
https://wskjniir5h.execute-api.ap-south-1.amazonaws.com/hello
Pretty-print
{
"message": "Hello from AWS Lambda!",
"application": "Python Web App",
"status": "success"
}
ENG
IN
06:32 PM
02-09-2026

GitHub Repository Preparation

Project folder:

aws-python-webapp/

```
├── app.py
├── requirements.txt
├── Dockerfile
├── templates/
│   └── index.html
└── README.md
```

Initialize Git

```
git init
git add .
git commit -m "Initial AWS Python web application project"
```

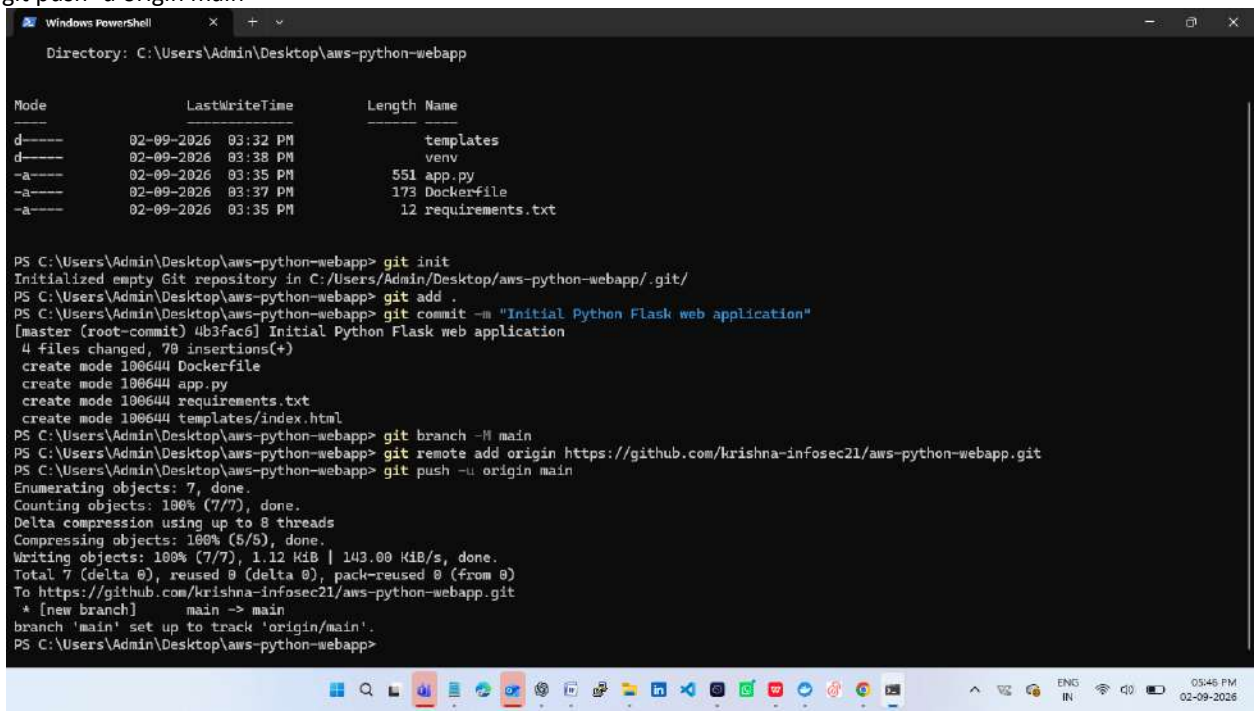
Create GitHub Repository

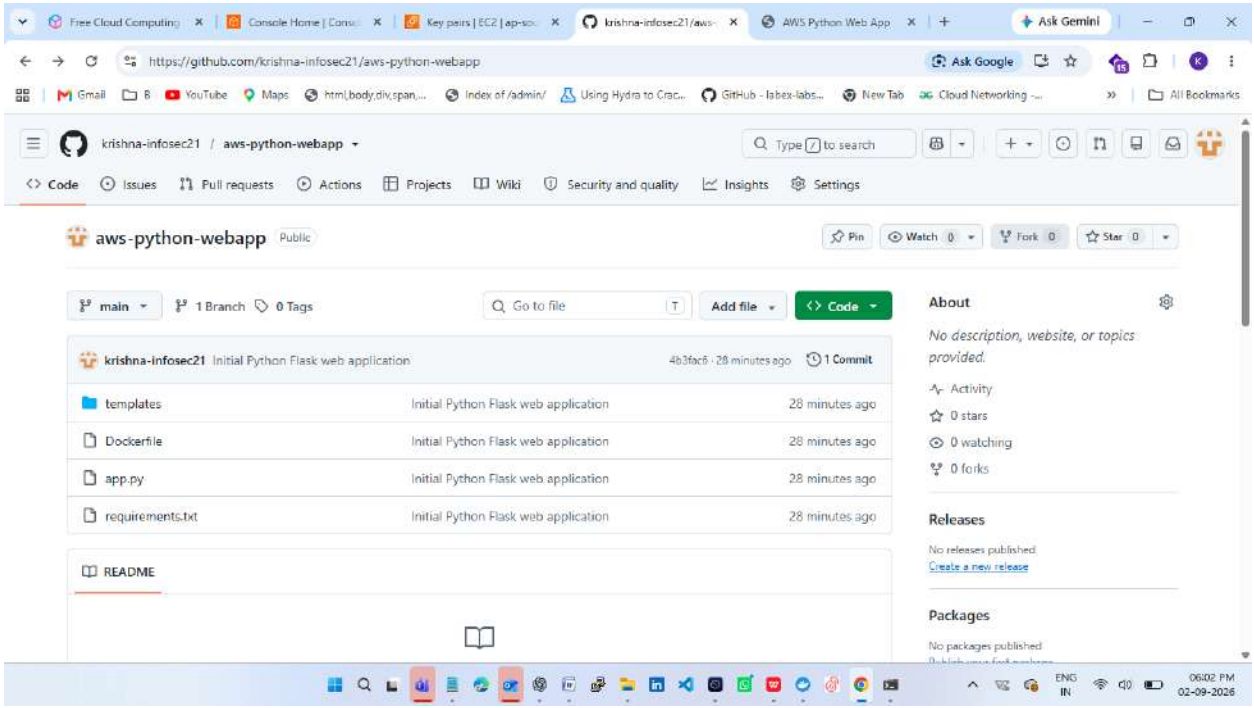
GitHub → **New Repository** → Repository name: aws-python-webapp

Create repository.

Then EC2 terminal:

git branch -M main
git remote add origin https://github.com/YOUR_USERNAME/aws-python-webapp.git
git push -u origin main




https://github.com/krishna-infosec21/aws-python-webapp
krishna-infosec21 / aws-python-webapp
Code
Issues
Pull requests
Actions
Projects
Wiki
Security and quality
Insights
Settings
aws-python-webapp
Public
main
1 Branch
0 Tags
Go to file
Add file
Code
krishna-infosec21 Initial Python Flask web application
4b3fac6 · 28 minutes ago
1 Commit
templates
Initial Python Flask web application
28 minutes ago
Dockerfile
Initial Python Flask web application
28 minutes ago
app.py
Initial Python Flask web application
28 minutes ago
requirements.txt
Initial Python Flask web application
28 minutes ago
README
About
No description, website, or topics provided.
Activity
0 stars
0 watching
0 forks
Releases
No releases published
Create a new release
Packages
No packages published

Testing Checklist

☐ Flask application works locally.

☐ Local /api/status returns success.

☐ EC2 deployment works.

☐ Docker image builds successfully.

☐ Docker container runs successfully.

☐ Docker image is available in ECR.

☐ ECS/Fargate service has one running task.

☐ ECS application response works.

☐ Lambda test returns HTTP 200.

☐ API Gateway GET /hello works.

☐ .gitignore is present.

☐ No AWS credentials, private keys, or secrets are in GitHub.

Project Outcome

The completed project demonstrates practical cloud deployment skills using Python Flask, Amazon EC2, Docker, Amazon ECR, Amazon ECS/Fargate, AWS Lambda, API Gateway,IAM and networking. The GitHub repository provides a reproducible source-code record, while this Word report provides the deployment procedure and screenshot evidence.